UNIVERSIDAD DE EL SALVADOR

Facultad Multidisciplinaria De Occidente

Departamento de Ingeniería y Arquitectura

Ingeniería en Desarrollo de Software

Asignatura:

Desarrollo y Reutilización de Software

Ciclo X / Quinto Año

Contenido:

Actividad Evaluada 2 – Tema 6
Sistema de Microservicios con Kafka para el Procesamiento de Datos en Tiempo Real
Documento Técnico del Proyecto | Can I Park?

Coordinadora de Cátedra:

Ing. Dinora Virginia Osorio de Melara

Tutora GT02:

Ing. Dinora Virginia Osorio de Melara

Alumnos:

Douglas Isaac Barrera Magaña - BM22025

Ricardo Enrique Heredia Ramos - HR21024

Gabriel Omar Calderón Calderón - CC22060

Douglas Enrique Sigüenza Quele - SQ22005

Fernando José Barraza Álvarez - BA22025

Repositorio:

https://github.com/Doisaac/can-i-park

Fecha:

25/10/26

Ingeniería en DESARROLLO DE SOFTWARE

Índice

Indice **2**
Introducción y enfoque del proyecto **3**
Problema, alcance y objetivos **4**
Arquitectura e interacción del sistema **5**
Decisiones de diseño tomadas por el equipo **7**
Reutilización, modularidad y mantenibilidad **9**
Implementación de los microservicios **10**
Control de Acceso 11
Disponibilidad 11
Notificaciones 12
Interfaz de demostración 13
Programación orientada a objetos, patrones y calidad del código **14**
Docker, ejecución e integración **15**
Pruebas, validación y evidencias **16**
Trabajo colaborativo y evolución del proyecto **17**
Conclusiones y mejoras futuras **19**

Introducción y enfoque del proyecto

El proyecto Can I Park? fue desarrollado como parte del Tema 6 de la Actividad Evaluada 2, denominado Sistema de Microservicios con Kafka para el Procesamiento de Datos en Tiempo Real. El reto consistía en construir una aplicación distribuida formada por microservicios independientes, comunicados mediante Apache Kafka y ejecutados a través de contenedores Docker. Desde las primeras reuniones del equipo entendimos que el propósito no era únicamente levantar varios servicios, sino demostrar que cada uno de ellos podía asumir una responsabilidad concreta y colaborar con los demás sin depender directamente de su implementación interna.

Como equipo decidimos trabajar con un sistema de disponibilidad de estacionamientos porque nos permitía representar un flujo sencillo de comprender y, al mismo tiempo, suficientemente completo para aplicar los conceptos de la asignatura. El funcionamiento parte de una acción cotidiana: un vehículo entra o sale de un estacionamiento. A partir de ese movimiento, el sistema debe determinar cuántos espacios se encuentran ocupados, cuántos permanecen disponibles y cuál es el estado general del lugar. Detrás de ese proceso aparentemente simple existe una cadena de recepción, transporte, procesamiento y reacción que resultaba adecuada para trabajar con microservicios y mensajería asíncrona.

Nuestra primera decisión importante fue evitar que el sistema se convirtiera en una aplicación grande dividida solamente por carpetas. Queríamos que la separación entre componentes tuviera una razón clara. Por esa razón Control de Acceso quedó responsable de recibir y publicar los movimientos, Disponibilidad asumió las reglas relacionadas con el estacionamiento y Notificaciones quedó encargado de reaccionar cuando el estado alcanzara una condición relevante. Apache Kafka se utilizó como intermediario entre estos servicios para evitar llamadas directas entre ellos.

También incorporamos una interfaz web. La intención de esta interfaz nunca fue sustituir Kafka ni crear una simulación visual desconectada del backend. La desarrollamos porque durante las pruebas observamos que mostrar el funcionamiento únicamente mediante varias consolas hacía más difícil seguir el recorrido de un movimiento. La interfaz se convirtió entonces en una herramienta para observar lo que realmente sucede en los servicios y facilitar la demostración del proyecto.

El desarrollo se realizó con Python 3.12, lenguaje consultado y autorizado para la actividad. FastAPI se utilizó para exponer las APIs, Pydantic para validar y estructurar los datos, confluent-kafka para trabajar con el broker, Nginx para servir la interfaz y Docker Compose para levantar la solución completa bajo una misma configuración.

Durante el proyecto intentamos mantener una forma de trabajo basada en preguntas concretas. Cada vez que una funcionalidad podía ubicarse en más de un servicio discutíamos cuál debía ser realmente su dueño. También analizamos qué partes tenían sentido como componentes reutilizables, qué ocurriría si Kafka entregaba nuevamente un mensaje, cuándo un movimiento podía considerarse procesado y cómo podíamos demostrar que el recorrido completo funcionaba. Varias de las decisiones que terminaron definiendo la arquitectura surgieron de esas discusiones y de problemas encontrados durante la integración.

Problema, alcance y objetivos

Can I Park? representa un estacionamiento que necesita mantener información actualizada sobre su disponibilidad. Cada vehículo se identifica mediante una placa y puede generar un movimiento de entrada o salida. A partir de esos movimientos se actualizan los vehículos presentes, la cantidad de espacios ocupados, la disponibilidad restante y el estado general del estacionamiento.

El problema no se limita a sumar o restar vehículos. El sistema también debe evitar situaciones inconsistentes. Una placa que ya se encuentra registrada no puede volver a entrar, una placa inexistente no puede generar una salida válida y no debe permitirse el ingreso de nuevos vehículos cuando la capacidad ya fue alcanzada. Estas reglas hicieron que el caso fuera útil para separar la recepción de la información de la lógica de negocio.

Definimos tres estados para representar de manera clara la situación del estacionamiento. El estado DISPONIBLE indica que la cantidad de espacios se encuentra por encima del umbral de advertencia. POCOS_ESPACIOS representa una situación en la que todavía hay capacidad, pero la disponibilidad es suficientemente baja como para generar una alerta. LLENO se utiliza cuando no queda ningún espacio disponible. La configuración normal del proyecto utiliza una capacidad de 100 espacios y un umbral de 10, aunque ambos valores se obtienen mediante variables de entorno y pueden cambiarse sin modificar la lógica.

El objetivo general del proyecto fue desarrollar una solución distribuida que mantuviera actualizada la disponibilidad del estacionamiento a partir de movimientos de entrada y salida, utilizando microservicios desacoplados, Apache Kafka para la comunicación asíncrona y Docker para una ejecución reproducible. A partir de ese objetivo acordamos separar la recepción de movimientos de las reglas que determinan la disponibilidad, compartir contratos entre productores y consumidores, evitar duplicación de lógica y comprobar el funcionamiento mediante pruebas aisladas y escenarios de extremo a extremo.

También definimos desde temprano qué funcionalidades no formarían parte de esta versión. Durante el desarrollo surgieron ideas relacionadas con reservas, pagos, autenticación, reconocimiento automático de placas, cámaras y múltiples estacionamientos. Todas podrían tener sentido en una aplicación real, pero consideramos que incorporarlas habría aumentado el tamaño del proyecto sin fortalecer directamente los conceptos que debíamos demostrar. Preferimos concentrar el trabajo en la arquitectura distribuida, Kafka, reutilización, modularidad, Docker y pruebas.

El estado del estacionamiento se mantiene en memoria. Esta fue una decisión consciente de alcance y no un intento de presentar la solución como un producto listo para producción. Una base de datos habría sido necesaria para conservar el estado después de un reinicio, pero no era un requisito del Tema 6 y habría añadido una capa adicional que no resultaba indispensable para demostrar el flujo principal. Como equipo preferimos que las partes esenciales quedaran correctamente implementadas, probadas y documentadas.

Arquitectura e interacción del sistema

La arquitectura de Can I Park? se diseñó alrededor de eventos. El recorrido comienza cuando el usuario registra un movimiento desde la interfaz o directamente desde Swagger. Esa solicitud llega mediante HTTP al microservicio de Control de Acceso, donde la placa y el tipo de movimiento son normalizados y validados antes de generar el evento correspondiente.

Una vez que los datos son válidos, Control de Acceso crea un MovimientoEvento y lo publica en el topic can-i-park.movimientos. A partir de ese momento deja de intervenir en la decisión relacionada con el estacionamiento. Esto fue importante para nosotros porque queríamos evitar que el primer servicio terminara acumulando responsabilidades que pertenecían al dominio.

Apache Kafka transporta el evento y Disponibilidad lo consume. Este microservicio conoce la capacidad, los vehículos registrados y las reglas de entrada y salida. Después de procesar el movimiento actualiza la ocupación, calcula los espacios disponibles y determina el estado general.

Durante el diseño discutimos si debía publicarse un segundo evento después de cada movimiento. Llegamos a la conclusión de que eso enviaría información innecesaria a Notificaciones. Una entrada puede cambiar la ocupación de 80 a 79 espacios sin modificar el estado DISPONIBLE, por lo que no existe una razón para generar una alerta. Decidimos entonces que Disponibilidad comparara el estado anterior con el resultado después del movimiento y publicara un EstadoEstacionamientoEvento únicamente cuando la categoría realmente cambiara.

Ese segundo evento se publica en can-i-park.estados. Notificaciones consume el mensaje y decide si debe generar una alerta. Si el estado es POCOS_ESPACIOS, crea una advertencia; si es LLENO, genera una alerta crítica. Cuando el estacionamiento vuelve a DISPONIBLE, el cambio también es recibido, pero no se agrega una nueva alerta.

La interacción completa puede entenderse como una cadena de responsabilidades. Control de Acceso recibe información externa y la transforma en un evento; Kafka transporta ese evento; Disponibilidad lo convierte en una decisión de negocio; Kafka transporta el cambio de estado; Notificaciones reacciona únicamente cuando la situación lo requiere. La interfaz se mantiene fuera de esa cadena de negocio y utiliza las APIs para registrar y consultar información.

Una regla que nos ayudó durante el desarrollo fue intentar explicar cada servicio en una sola frase. Control de Acceso recibe y publica. Disponibilidad interpreta y decide. Notificaciones reacciona. Cuando una nueva función no encajaba claramente en alguna de esas responsabilidades, revisábamos si realmente debía existir allí.

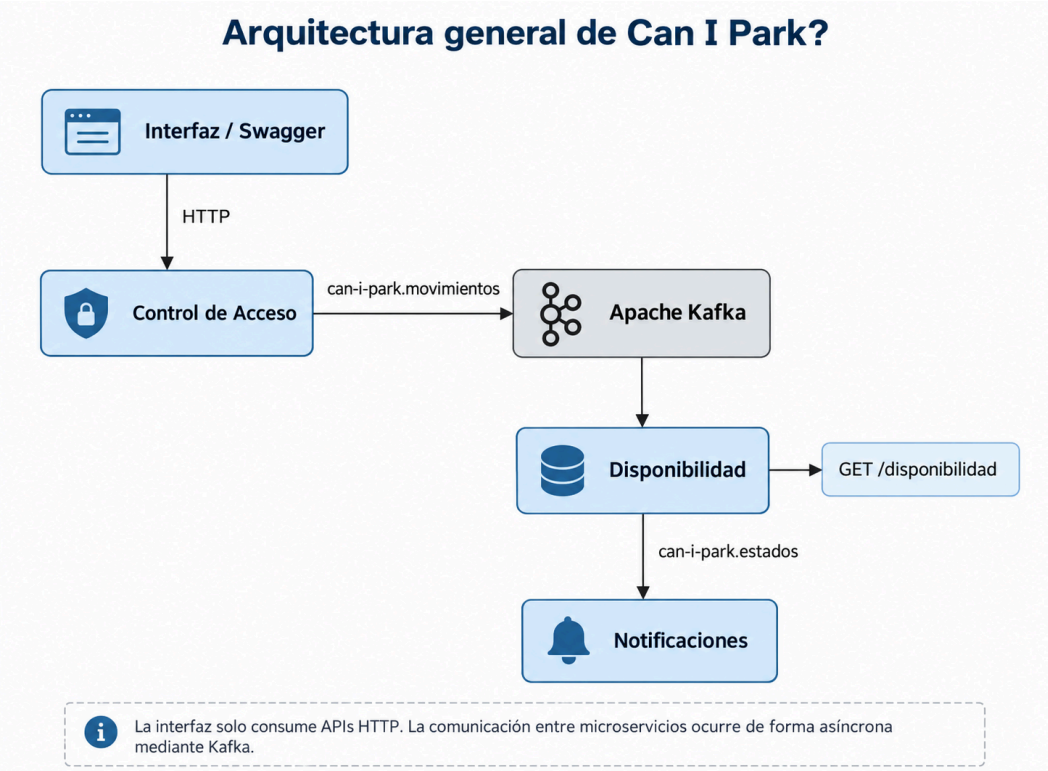


Figura 1. Arquitectura general de Can I Park?

Esta arquitectura permitió que los servicios evolucionaran con menor dependencia entre sí. Control de Acceso no necesita saber cuántos espacios existen y Notificaciones no necesita conocer cómo se valida una entrada. Los contratos intercambiados mediante Kafka son el punto de acuerdo entre componentes.

Decisiones de diseño tomadas por el equipo

Una de las primeras decisiones que discutimos fue qué debía significar para el usuario que Control de Acceso recibiera correctamente una solicitud. Debido a que el procesamiento continúa después de la respuesta HTTP, consideramos que utilizar 200 OK podía transmitir una idea incorrecta. En ese momento todavía no sabemos si Disponibilidad aceptará o rechazará el movimiento. Lo único confirmado es que Control de Acceso validó la información y Kafka recibió el evento. Por esa razón decidimos utilizar 202 Accepted, que representa mejor el comportamiento asíncrono.

Esta decisión generó otra necesidad: identificar qué ocurre posteriormente con una solicitud que ya recibió un 202. Para resolverlo asignamos un evento_id único a cada

movimiento. Disponibilidad conserva el resultado asociado a ese identificador y expone una consulta que permite saber si el evento ya fue procesado. La interfaz utiliza el mismo identificador para relacionar la solicitud original con el resultado obtenido después de pasar por Kafka.

El evento_id también resultó útil para manejar reentregas. Cuando analizamos cómo funciona la mensajería comprendimos que un consumidor puede volver a recibir un evento. Si una entrada válida se aplicara dos veces, la ocupación podría quedar incorrecta. Para evitarlo, Disponibilidad conserva temporalmente los resultados de eventos procesados. Si recibe nuevamente un identificador conocido, devuelve el resultado anterior sin modificar otra vez el estado.

También decidimos desactivar el auto commit de los consumidores. No queríamos que Kafka marcara un mensaje como atendido antes de saber si la lógica terminó correctamente. El flujo quedó planteado de manera que primero se lee el mensaje, luego se deserializa y procesa, y solamente después se confirma el offset. Si ocurre un error antes de completar el proceso, el commit no se ejecuta.

Esta decisión implica que un mensaje podría volver a entregarse, pero precisamente por eso la idempotencia del dominio forma parte del diseño. Como equipo vimos que ambas decisiones se complementaban: el commit manual evita confirmar prematuramente y el evento_id protege el dominio si ocurre una reentrega.

Otra decisión importante fue utilizar un set para mantener los vehículos dentro del estacionamiento. Una primera solución podría haber consistido simplemente en guardar un contador, pero eso no permitía saber qué placas formaban parte de la ocupación. Necesitábamos comprobar si un vehículo ya estaba dentro antes de aceptar una entrada y verificar su existencia antes de procesar una salida. El conjunto nos permitió resolver ambas necesidades y evitar duplicados.

La configuración de capacidad y umbral tampoco quedó escrita directamente dentro de las reglas. Preferimos obtener esos valores mediante variables de entorno. Así, la misma implementación puede trabajar con diferentes capacidades sin tener que modificar el código. Durante las pruebas E2E usamos temporalmente capacidades pequeñas para alcanzar rápidamente POCOS_ESPACIOS o LLENO, mientras que la configuración normal permanece en 100/10.

También discutimos cuándo debía producirse un evento de notificación. Enviar un mensaje después de cada movimiento habría sido técnicamente posible, pero no aportaba valor. Preferimos publicar solamente cuando cambia el estado general. Esta decisión evita ruido, reduce mensajes innecesarios y mantiene Notificaciones enfocado en cambios relevantes.

La interfaz y Nginx también surgieron de decisiones prácticas. No queríamos que el navegador se conectara directamente con Kafka ni que conociera las direcciones internas de los contenedores. Nginx sirve los archivos estáticos y funciona como proxy hacia las APIs. Con esto conseguimos mantener la interfaz como una capa de observación y no como un participante directo de la mensajería.

Al revisar estas decisiones en conjunto observamos que muchas no fueron independientes. La asincronía llevó al uso de 202; el 202 hizo necesario el seguimiento mediante evento_id; las reentregas llevaron a la idempotencia; el commit manual reforzó esa necesidad; y la separación de responsabilidades nos llevó a publicar solo cambios de estado. La arquitectura fue tomando forma a medida que resolvíamos estos problemas.

Reutilización, modularidad y mantenibilidad

La reutilización fue uno de los aspectos que más cuidamos durante el desarrollo, especialmente cuando comenzamos a conectar productores y consumidores. Cuando empezamos a conectar los servicios nos dimos cuenta de que productores y consumidores necesitarían exactamente las mismas definiciones de eventos. Si cada integrante mantenía una versión diferente, un cambio aparentemente pequeño podía romper la comunicación.

A partir de ese problema decidimos crear canipark_shared. La intención fue tener un paquete común que contuviera únicamente elementos que realmente debían ser reutilizados por más de un servicio. Allí incluimos contratos como MovimientoEntrada, MovimientoEvento y ResultadoMovimiento. También compartimos EstadoEstacionamientoEvento, TipoMovimiento y EstadoEstacionamiento.

La principal ventaja es que el productor y el consumidor trabajan sobre la misma definición. Control de Acceso crea un MovimientoEvento y Disponibilidad interpreta ese mismo modelo. Disponibilidad crea un EstadoEstacionamientoEvento y Notificaciones utiliza la misma estructura para consumirlo. Con esto evitamos tener copias del contrato que pudieran evolucionar de forma diferente.

La creación de shared también nos obligó a decidir qué elementos debían permanecer fuera. No queríamos convertirlo en una carpeta donde terminara toda función que pudiera usarse más de una vez. GestorDisponibilidad, por ejemplo, pertenece al negocio de Disponibilidad. Aunque técnicamente podría importarse desde otro lugar, moverlo a shared habría debilitado la separación del dominio.

Como equipo adoptamos un criterio: los contratos y las utilidades transversales pueden compartirse; las reglas propias de cada servicio deben permanecer junto al servicio que las posee. Esta regla nos ayudó a mantener el paquete común pequeño y con un propósito fácil de entender.

Además de los contratos, encontramos reutilización en la forma de trabajar con Kafka. Los servicios necesitaban crear productores, consumidores, serializar JSON, procesar mensajes y confirmar offsets. Si cada componente implementaba estas operaciones por separado, era posible terminar con configuraciones distintas. Decidimos entonces centralizar estas tareas en utilidades compartidas.

Para nosotros, reutilizar no significó solamente ahorrar líneas de código. También significó mantener una política consistente de comunicación. El mismo mecanismo de serialización, publicación y consumo puede utilizarse en distintos servicios sin duplicar configuración.

La modularidad se observa en que los microservicios no importan directamente la lógica interna de otros servicios. Control de Acceso no necesita acceder a GestorDisponibilidad y Notificaciones no necesita saber cómo se validan las placas. Cada uno trabaja con los contratos acordados.

Esta separación también favorece la mantenibilidad. Si en el futuro cambia el umbral para determinar POCOS_ESPACIOS, la modificación debe concentrarse en Disponibilidad. Si las alertas necesitan enviarse por correo o por una aplicación móvil, el cambio puede concentrarse en Notificaciones. Si aparece otra fuente de movimientos, podría publicar el mismo contrato en Kafka.

Consideramos que esta capacidad de modificar una parte con menor impacto en las demás es una evidencia más valiosa de modularidad que simplemente tener muchos archivos o carpetas.

Implementación de los microservicios

Control de Acceso

Control de Acceso representa el inicio del procesamiento. Recibe una placa y un tipo de movimiento mediante POST /movimientos. Antes de generar el evento, los datos son normalizados y validados con Pydantic.

Durante la implementación encontramos un detalle que parecía pequeño pero afectaba el resultado: el orden de normalización. Decidimos convertir la placa a mayúsculas, eliminar espacios y comprobar los caracteres permitidos antes de evaluar su longitud. De esta forma una entrada con espacios alrededor no puede parecer válida únicamente porque esos espacios aumentan artificialmente la cantidad de caracteres.

Una vez validados los datos se genera el MovimientoEvento. El evento contiene la información recibida, un UUID utilizado como evento_id y la fecha de ocurrencia. El movimiento se publica en can-i-park.movimientos, utilizando la placa como clave del mensaje.

Cuando Kafka confirma correctamente la publicación, el servicio responde 202 Accepted. Si la entrega falla, se registra el problema y se devuelve 503 Service Unavailable. Esta decisión evita informar al usuario que la solicitud fue aceptada cuando realmente el broker nunca confirmó el mensaje.

También procuramos que la ruta HTTP no dependiera directamente de una clase concreta de Kafka. Para ello utilizamos PublicadorMovimientos, definido mediante Protocol. La ruta necesita solamente un componente capaz de publicar un evento. En ejecución se utiliza PublicadorKafka y en pruebas puede utilizarse un objeto falso compatible con la misma interfaz.

Esta separación facilitó las pruebas y nos permitió mantener la infraestructura fuera de la lógica principal del endpoint.

Disponibilidad

Disponibilidad es el microservicio que contiene la mayor parte de las reglas del dominio. Allí se encuentran la capacidad, las placas registradas, la ocupación, los espacios disponibles y los resultados de eventos procesados.

Cuando llega una entrada, el servicio comprueba primero si la placa ya está registrada. Si existe, el movimiento se rechaza y el estado no cambia. Después se verifica la capacidad. Si el estacionamiento está lleno, tampoco se acepta el ingreso. Solamente cuando ambas condiciones se cumplen se agrega la placa.

Para una salida se comprueba que el vehículo exista. Una salida de una placa que no se encuentra dentro se considera inválida y no altera la ocupación. Cuando la placa sí está registrada, se elimina y se recalcula el estado.

El cálculo del estado depende de los espacios disponibles. Si la cantidad llega a cero, el resultado es LLENO. Si continúa por encima de cero pero es menor o igual al umbral, el estado pasa a POCOS_ESPACIOS. En cualquier otra situación se mantiene DISPONIBLE.

Disponibilidad también conserva resultados recientes por evento_id. Antes de ejecutar una regla revisa si el evento fue procesado anteriormente. Si ya existe, devuelve el mismo resultado y finaliza sin modificar los vehículos ni publicar otra transición. Esta protección mantiene consistente el dominio ante reentregas.

La información se protege mediante mecanismos adecuados para el acceso concurrente, ya que el worker Kafka puede modificar el estado mientras las rutas HTTP reciben consultas. Esta fue otra razón por la que preferimos encapsular las reglas dentro de un gestor en lugar de mantener variables dispersas en la API.

Notificaciones

Notificaciones forma parte del flujo final, pero su responsabilidad es más pequeña. Decidimos que este servicio consumiera cambios de estado y no movimientos crudos. De esa forma recibe una decisión que ya fue tomada por Disponibilidad y no necesita repetir reglas relacionadas con capacidad o vehículos.

Cuando recibe POCOS_ESPACIOS, genera una alerta indicando la cantidad disponible. Cuando recibe LLENO, registra una alerta crítica. Si el estado vuelve a DISPONIBLE, procesa la transición sin crear una nueva alerta.

El servicio mantiene un historial limitado en memoria. Esta decisión fue suficiente para el alcance académico y evita un crecimiento ilimitado durante la ejecución.

La interfaz distingue entre una alerta histórica y una alerta activa. Un estacionamiento puede haber estado lleno anteriormente y conservar ese registro, pero si actualmente vuelve a estar disponible, la pantalla muestra que no existen alertas activas.

Interfaz de demostración

La interfaz surgió de una necesidad que identificamos durante las pruebas: mostrar el funcionamiento del sistema de una forma más amigable. Al principio verificábamos cada servicio desde terminales diferentes, lo que hacía difícil explicar el recorrido a una persona que no conociera la arquitectura.

Decidimos crear una interfaz sencilla que mostrara el estado actual y el progreso del movimiento. Desde allí se puede registrar una entrada o salida, consultar capacidad, ocupación, disponibilidad, vehículos y alertas.

El evento_id aparece durante el proceso porque permite relacionar visualmente la solicitud con el resultado que genera Disponibilidad. La interfaz consulta el backend hasta obtener ese resultado y no utiliza una espera fija para suponer que Kafka terminó.

Nginx sirve los archivos estáticos y actúa como proxy hacia las APIs. Las rutas /api/acceso, /api/disponibilidad y /api/notificaciones permiten que el navegador trabaje dentro del mismo origen sin conocer directamente las direcciones internas de Docker.

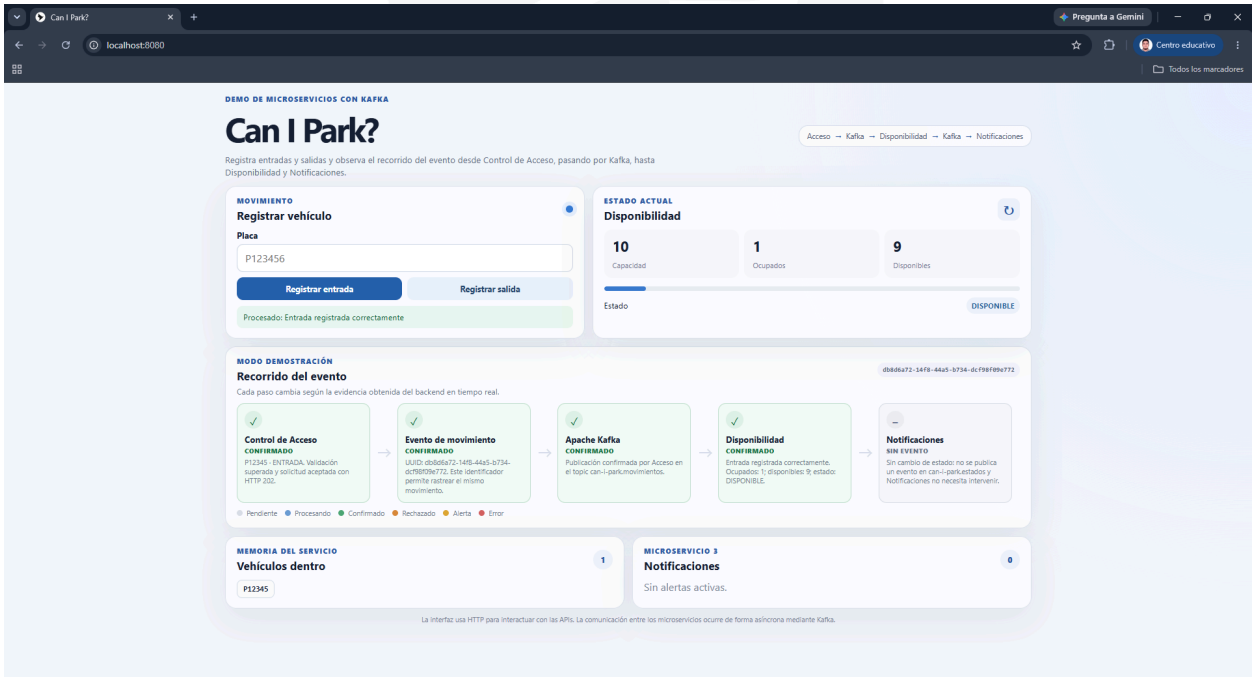


Figura 2. Interfaz final de Can I Park?

Programación orientada a objetos, patrones y calidad del código

Durante el desarrollo también cuidamos la aplicación de conceptos de programación orientada a objetos, procurando utilizar clases cuando representaban responsabilidades reales. No queríamos crear jerarquías únicamente para aumentar la cantidad de objetos del proyecto.

GestorDisponibilidad encapsula el estado del estacionamiento y las operaciones que lo modifican. PublicadorKafka concentra el comportamiento relacionado con la publicación. Los workers administran el consumo de mensajes y los modelos Pydantic representan contratos tipados.

El uso de Protocol también resultó importante. PublicadorMovimientos permite trabajar contra una abstracción. El endpoint no necesita conocer si detrás existe Kafka o una implementación falsa utilizada durante una prueba. En Python esto permite aplicar polimorfismo estructural sin construir una jerarquía de herencia que el problema no necesita.

En cuanto a patrones de diseño, preferimos documentar aquellos que realmente aparecen en la solución. El flujo sigue claramente el modelo Producer/Consumer. Control de Acceso produce movimientos, Disponibilidad los consume y a su vez produce cambios de estado, y Notificaciones consume esos cambios.

Kafka también permite trabajar bajo un esquema Publish/Subscribe, porque los productores publican eventos sin conocer directamente qué componente concreto los consumirá. Este modelo comparte con Observer la idea de reaccionar ante cambios, aunque en nuestro proyecto ocurre de forma distribuida mediante un broker.

La inversión de dependencias aparece en el uso de PublicadorMovimientos. FastAPI facilita además la inyección de dependencias, lo que permite sustituir el publicador durante una prueba sin modificar el endpoint.

Toda la arquitectura intenta mantener alta cohesión y bajo acoplamiento. Las funciones relacionadas con una misma responsabilidad se mantienen juntas y los servicios se comunican mediante contratos en lugar de depender de detalles internos.

Decidimos no forzar MVC, Singleton u otro patrón solamente para poder mencionarlo en el informe. Consideramos que habría sido más difícil defender una implementación artificial que explicar con claridad los patrones que el problema realmente necesitaba.

La calidad del código también se trabajó desde la organización. Los archivos main.py se concentran principalmente en las APIs y el ciclo de vida, mientras que las reglas se mantienen

fuera de las rutas HTTP. Los workers conectan Kafka con el dominio, la configuración se obtiene externamente y los contratos compartidos permanecen dentro de canipark_shared.

Los nombres de clases y modelos se relacionan con conceptos del proyecto. Preferimos GestorDisponibilidad, MovimientoEvento o EstadoEstacionamiento en lugar de nombres genéricos. Los comentarios se utilizaron solamente cuando una decisión no resulta evidente leyendo el código.

Docker, ejecución e integración

Docker fue un requisito del Tema 6, pero durante el desarrollo terminó resolviendo también un problema práctico del equipo. Cada microservicio posee su propio Dockerfile y Docker Compose se encarga de levantar Kafka, Control de Acceso, Disponibilidad, Notificaciones e Interfaz bajo una misma red.

Durante las pruebas encontramos diferencias entre las instalaciones locales de Python. Una computadora podía resolver una dependencia o un comando de una forma distinta a otra. Esto generaba incertidumbre porque una falla podía provenir del código o del entorno. Para reducir esa diferencia decidimos crear un contenedor auxiliar de pruebas basado en Python 3.12. Este contenedor no representa otro microservicio de negocio. Su única responsabilidad es proporcionar un entorno reproducible para ejecutar validaciones.

La integración también nos mostró que un contenedor puede aparecer como iniciado y no estar realmente preparado. Por ese motivo agregamos healthchecks y dependencias de arranque. Kafka posee una verificación propia y las APIs exponen GET /health. Los servicios que dependen del broker esperan a que este alcance una condición saludable antes de comenzar a trabajar.

Uno de los problemas que encontramos apareció en el healthcheck de la interfaz. Nginx se encontraba funcionando, pero la comprobación original mediante localhost dentro del contenedor no reflejaba correctamente su estado. Probamos la misma petición utilizando 127.0.0.1, confirmamos que el servidor respondía y modificamos la validación.

Aunque este ajuste era pequeño, nos ayudó a comprender que la integración tiene detalles que no aparecen cuando cada componente se revisa de manera aislada.

Para levantar la solución completa se utiliza docker compose up -d --build. Después puede ejecutarse docker compose ps para verificar el estado de los componentes. La interfaz se

publica en http://localhost:8080, mientras que los Swagger de Control de Acceso, Disponibilidad y Notificaciones se encuentran en los puertos 8001, 8002 y 8003 respectivamente.

Pruebas, validación y evidencias

Desde el inicio consideramos que una demostración manual no era suficiente para afirmar que el proyecto funcionaba. Por esa razón trabajamos con diferentes niveles de prueba.

Las pruebas automatizadas permiten validar reglas, contratos, APIs y utilidades de forma rápida. La versión final cuenta con 26 pruebas aprobadas. Entre los escenarios cubiertos se encuentran normalización de placas, respuestas de validación, errores de publicación, utilidades Kafka, commit manual, configuración, reglas de entrada y salida, capacidad, estados, idempotencia, consultas de Disponibilidad y generación de alertas.

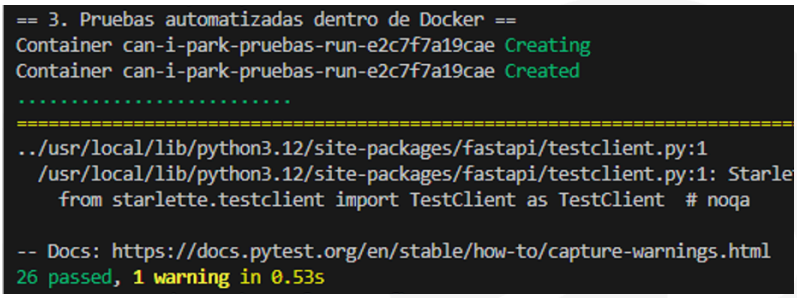


Figura 3. Resultado de pruebas automatizadas: 26 passed.

Las pruebas aisladas no demostraban por sí solas que Kafka estuviera transportando mensajes entre servicios. Para comprobar esa integración se utilizaron escenarios E2E.

El primer flujo verifica Control de Acceso → Kafka → Disponibilidad. El script registra una placa única mediante la API, obtiene el evento_id, espera hasta que Disponibilidad procese el movimiento y confirma que la ocupación cambió. Después registra la salida y comprueba que el estado regrese al punto esperado.

El segundo flujo verifica Disponibilidad → Kafka → Notificaciones. Para alcanzar una transición rápidamente se utiliza temporalmente una capacidad pequeña. Los movimientos provocan el cambio a POCOS_ESPACIOS, Disponibilidad publica en can-i-park.estados y Notificaciones debe recibir el evento y generar la alerta. Al terminar, la configuración se restaura.

Para el cierre del proyecto quisimos que toda la validación pudiera repetirse siguiendo un único procedimiento. Por esa razón incorporamos scripts para PowerShell y Bash. El validador ejecuta las pruebas dentro del contenedor auxiliar, levanta la solución, ejecuta ambos E2E, verifica la interfaz, revisa los componentes y restaura la configuración normal.

La ejecución final obtuvo 26 pruebas aprobadas, ambos flujos E2E correctos y la interfaz verificada. El proceso terminó con el mensaje VALIDACION COMPLETA: OK.

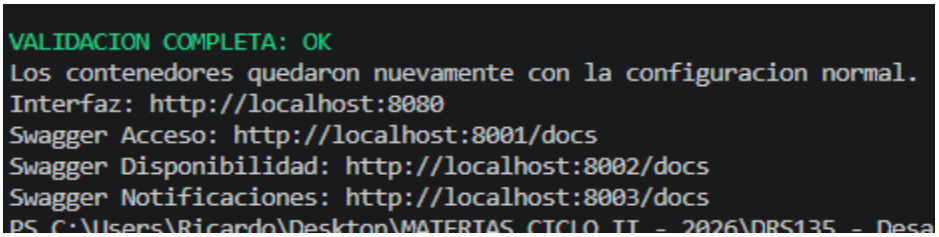


Figura 4. Resultado final del validador

Después de la validación se restauraron la capacidad de 100 espacios y el umbral de 10. Tras esperar el tiempo necesario para los healthchecks, Kafka, Control de Acceso, Disponibilidad, Notificaciones e Interfaz quedaron en estado healthy.

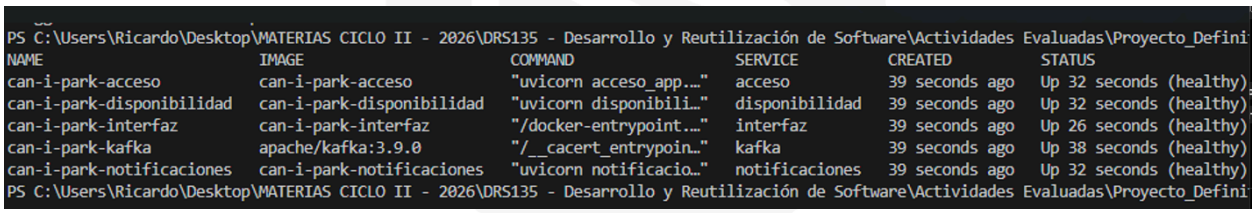


Figura 5. docker compose ps con los cinco componentes healthy.

Decidimos conservar estas evidencias dentro del repositorio porque permiten que la persona evaluadora compruebe los resultados sin depender únicamente de lo escrito en este informe.

Trabajo colaborativo y evolución del proyecto

Can I Park? fue desarrollado por cinco integrantes. Al comienzo distribuimos áreas principales para que todos pudiéramos avanzar en paralelo sin modificar constantemente los mismos archivos.

Douglas Isaac Barrera Magaña trabajó principalmente sobre Control de Acceso, Gabriel Omar Calderón Calderón sobre Disponibilidad, Douglas Enrique Sigüenza Quele sobre Notificaciones, Fernando José Barraza Álvarez sobre la interfaz y documentación asociada, y Ricardo Enrique Heredia Ramos sobre componentes compartidos, contratos, Kafka e integración.

Esta distribución funcionó como punto de partida, pero el proyecto no terminó siendo la unión de cinco trabajos independientes. Cada servicio dependía de contratos comunes y tenía que integrarse con los demás. Una modificación en un evento podía afectar al productor y al consumidor, mientras que un cambio en Docker podía afectar la ejecución completa.

Por esa razón utilizamos ramas y pull requests para integrar los componentes de manera controlada. develop se utilizó como rama de integración y main quedó reservada para la versión estable.

Durante las revisiones pudimos identificar cambios que necesitaban ajustes antes de llegar a la rama principal. También manteníamos trazabilidad sobre qué componente se estaba incorporando y quién debía revisar cada integración.

La etapa más colaborativa fue el cierre. Los E2E, la interfaz, los healthchecks y la validación completa no pertenecían únicamente a un microservicio. Era necesario comprobar cómo funcionaba la solución como un todo.

Varias decisiones surgieron justamente en esta fase. La necesidad de canipark_shared apareció cuando vimos que productores y consumidores empezaban a repetir contratos. El seguimiento mediante evento_id tomó importancia cuando empezamos a observar el procesamiento asíncrono desde la interfaz. La idempotencia se volvió necesaria al analizar posibles reentregas. Los healthchecks aparecieron al comprobar que un contenedor iniciado no necesariamente estaba listo. El runner de pruebas nació por las diferencias entre entornos locales.

La interfaz también surgió de una necesidad del grupo. Durante las primeras pruebas teníamos que abrir Swagger, terminales y logs para explicar el recorrido. Consideramos que la demostración debía ser más fácil de seguir y decidimos crear una vista que mostrara información real sin ocultar la arquitectura.

Este proceso hizo que el diseño evolucionara. No todas las decisiones estaban resueltas en el primer día y consideramos que eso forma parte del aprendizaje del proyecto. Algunas mejoras aparecieron porque una idea inicial no era suficiente cuando todos los componentes comenzaron a interactuar.

Conclusiones y mejoras futuras

Al finalizar el desarrollo de Can I Park?, consideramos que logramos construir una solución que representa de forma clara los conceptos principales trabajados en la actividad. El sistema quedó dividido en microservicios con responsabilidades definidas, comunicados mediante Kafka y ejecutados con Docker. Más que separar el proyecto en diferentes componentes, buscamos que cada uno tuviera una función concreta dentro del recorrido de la información.

Una de las decisiones que más aportó al proyecto fue mantener separados Control de Acceso, Disponibilidad y Notificaciones. Control de Acceso recibe y publica los movimientos, Disponibilidad concentra las reglas del estacionamiento y Notificaciones reacciona únicamente cuando ocurre un cambio relevante. Esta organización nos ayudó a evitar que una misma lógica terminara repetida en diferentes partes del sistema y facilitó la integración entre los componentes.

Durante el desarrollo también comprendimos que trabajar con Kafka implicaba considerar situaciones que no aparecen de la misma manera en una aplicación síncrona. El uso de 202 Accepted, el seguimiento mediante evento_id, la idempotencia y el commit manual surgieron al analizar qué podía ocurrir mientras un mensaje era transportado y procesado. Estas decisiones no fueron incorporadas solamente por una cuestión técnica; aparecieron como respuesta a problemas que fuimos identificando durante las pruebas y la integración.

La reutilización tuvo un proceso similar. Cuando observamos que los servicios necesitaban compartir contratos y configuraciones, decidimos crear canipark_shared. Al mismo tiempo, acordamos que las reglas propias de cada microservicio debían permanecer dentro de su dominio. Esta separación nos permitió reutilizar lo que realmente debía ser común sin convertir el paquete compartido en una dependencia innecesaria para todo el sistema.

Las pruebas nos permitieron cerrar el proyecto con mayor seguridad. La versión final obtuvo 26 pruebas automatizadas aprobadas, además de dos escenarios E2E que comprobaron el recorrido real de los mensajes mediante Kafka. También se verificó la interfaz, se restauró la configuración normal de capacidad 100 y umbral 10 y los cinco componentes principales quedaron en estado healthy. Para nosotros, estas evidencias fueron importantes porque permitieron comprobar el comportamiento del sistema más allá de una demostración manual.

Como equipo también aprendimos que varios de los problemas más importantes aparecen cuando los componentes comienzan a trabajar juntos. Los healthchecks, el contenedor de pruebas y algunos ajustes de integración surgieron precisamente en esa etapa. Esto nos hizo revisar decisiones que parecían correctas cuando los servicios se ejecutaban por separado y reforzó la importancia de validar la solución completa.

Can I Park? todavía puede evolucionar. Una versión posterior podría incorporar persistencia para conservar el estado después de un reinicio, soporte para varios estacionamientos, autenticación, un historial permanente de notificaciones y una estrategia más completa para mensajes que no puedan procesarse. Decidimos no incorporar estas funciones en esta entrega porque preferimos mantener un alcance que pudiéramos implementar, probar y explicar correctamente.

Como conclusión, consideramos que el mayor aprendizaje del proyecto fue entender que una arquitectura modular no depende solamente de dividir el código. Fue necesario definir responsabilidades, contratos y formas de comunicación que permitieran que cada parte funcionara sin conocer los detalles internos de las demás. El resultado final refleja tanto la implementación técnica como las decisiones que fuimos tomando entre todos para mantener una solución organizada, reutilizable y funcional.